e. @DeclareRoles("ADMIN")

8.3 Attribute-based access control

While RBAC is a very successful access control model that has been widely deployed, in many cases the desired access control policies cannot be expressed through simple role assignments. Consider the call center agent example from section 8.2.4. As well as preventing the agent from accessing customer records outside of their contracted working hours, you might also want to prevent them accessing those records if they are not actually on a call with that customer. Allowing each agent to access all customer records during their working hours is still more authority than they really need to get their job done, violating the principle of least privilege. It may be that you can determine which customer the call agent is talking to from their phone number (caller ID), or perhaps they enter an account number using the keypad before they are connected to an agent. You'd like to only allow the agent access to just that customer's file for the duration of the call, perhaps allowing 5 minutes afterward for them to finishing writing any notes.

To handle these kinds of dynamic access control decisions, an alternative to RBAC has been developed known as ABAC: attribute-based access control. In ABAC, access control decisions are made dynamically for each API request using collections of attributes grouped into four categories: